

**Università Politecnica delle Marche**

**Facoltà di Ingegneria**

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

---



**Computer Graphics**

**Document your reality**

**Professori**

Prof. Primo Zingaretti  
Prof. Lorenzo Stacchio  
Prof. Claudio Sirocchi

**Studenti**

Loris Bottegoni  
Leonardo Cambiotti  
Valerio Crocetti

---

**Anno Accademico 2025-2026**

# Indice

|          |                                                                           |          |
|----------|---------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                                                       | <b>2</b> |
| 1.1      | Obiettivi e Task di Progetto                                              | 2        |
| 1.2      | Flessibilità di Deployment e Ambito di Esecuzione                         | 2        |
| 1.3      | Struttura del Documento                                                   | 2        |
| <b>2</b> | <b>Architettura del Sistema</b>                                           | <b>4</b> |
| 2.1      | Topologia Client-Server e Pipeline di Scansione                           | 4        |
| 2.2      | Sistema di Discovery Dinamico su UDP                                      | 5        |
| 2.2.1    | Protocollo di Handshake e Risoluzione del Peer                            | 6        |
| 2.2.2    | Instradamento Multi-Interfaccia e Kernel Routing                          | 6        |
| 2.3      | Stack Tecnologico                                                         | 6        |
| 2.3.1    | Client e Frontend Immersivo (Unity Engine)                                | 6        |
| 2.3.2    | Backend Computazionale e Middleware                                       | 7        |
| 2.3.3    | Motore di Visione e Grounding Multimodale (MLLM)                          | 7        |
| <b>3</b> | <b>Sviluppo del Backend: Intelligenza Artificiale e Logica di Dominio</b> | <b>8</b> |
| 3.1      | API e Gestione delle Richieste                                            | 8        |
| 3.1.1    | Architettura degli Endpoint REST                                          | 8        |
| 3.1.2    | Concorrenza e Isolamento delle Risorse                                    | 8        |
| 3.1.3    | Elaborazione in Background e Gestione Errori                              | 9        |
| 3.2      | Integrazione con il Modello MLLM                                          | 9        |
| 3.2.1    | Normalizzazione Ottica e Pre-elaborazione del Frame                       | 9        |
| 3.2.2    | Parsing Incrementale dello Stream JSON                                    | 9        |
| 3.2.3    | Terminazione Anticipata dello Stream                                      | 9        |
| 3.3      | Prompt Engineering e Regole di Estrazione                                 | 10       |
| 3.3.1    | Dicotomia Operativa: Shape A e Shape B                                    | 10       |
| 3.3.2    | Grounding Spaziale con Vertici Nominati e Prevenzione delle Trasposizioni | 10       |
| 3.3.3    | Verifica Empirica tramite Pattern Anti-Diagonale                          | 10       |
| 3.3.4    | Prioritizzazione e Conservazione delle Nuove Informazioni                 | 11       |
| 3.4      | Logica di Re-identificazione e Persistenza Semantica                      | 11       |
| 3.4.1    | Pipeline di Risoluzione dell'Identità                                     | 11       |
| 3.4.2    | Filtro delle Annotazioni Ridondanti                                       | 12       |
| 3.4.3    | Distillazione della Memoria Evolutiva (EntityMemory)                      | 12       |

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Gestione della Memoria e dello Storage</b>                                   | <b>13</b> |
| 4.1      | Persistenza dei Dati e Struttura dello Storage . . . . .                        | 13        |
| 4.1.1    | Organizzazione del File System e Indici in Memoria . . . . .                    | 13        |
| 4.1.2    | Partizionamento degli Ambienti . . . . .                                        | 13        |
| 4.1.3    | Struttura dei Dati: Entità e Bounding Box . . . . .                             | 13        |
| 4.1.4    | Struttura delle Annotazioni e Salvataggio delle Acquisizioni Visive . . . . .   | 14        |
| 4.2      | Distillazione della Memoria Semantica (Memory System) . . . . .                 | 15        |
| 4.2.1    | Il Ciclo di Sintesi Incrementale . . . . .                                      | 15        |
| 4.2.2    | Grafo di Relazione Semantica e Ordinamento per Sezioni . . . . .                | 15        |
| <b>5</b> | <b>Sviluppo del Frontend Unity: Gestione Frame e Interfaccia</b>                | <b>17</b> |
| 5.1      | Architettura delle Sorgenti dei Frame (Frame Sources) . . . . .                 | 17        |
| 5.1.1    | Astrazione e Modello Pinhole . . . . .                                          | 17        |
| 5.1.2    | Modalità Passthrough e Meta Passthrough Camera API . . . . .                    | 17        |
| 5.1.3    | Modalità Sintetica VR e Modalità Video Offline . . . . .                        | 18        |
| 5.1.4    | Analisi della Nitidezza . . . . .                                               | 18        |
| 5.2      | HUD Immersivo e Sistema di Diagnostica . . . . .                                | 19        |
| 5.2.1    | HUD Spaziale Flottante . . . . .                                                | 19        |
| 5.3      | Unificazione e Normalizzazione degli Input . . . . .                            | 19        |
| 5.3.1    | Mappatura delle Azioni di Scansione e Controllo . . . . .                       | 19        |
| 5.3.2    | Gestione dell'Hand Tracking e Isolamento delle Gesture di Sistema . . . . .     | 20        |
| <b>6</b> | <b>Spatial Computing e Posizionamento AR</b>                                    | <b>21</b> |
| 6.1      | Dal 2D al 3D: Raycasting e Geometria Proiettiva . . . . .                       | 21        |
| 6.1.1    | Proiezione Conforme e Disaccoppiamento Temporale . . . . .                      | 21        |
| 6.1.2    | Campionamento a Corone Concentriche e Votazione a Maggioranza . . . . .         | 21        |
| 6.2      | Integrazione con Meta MRUK (Mixed Reality Utility Kit) . . . . .                | 22        |
| 6.2.1    | Scene Understanding e Global Mesh . . . . .                                     | 22        |
| 6.2.2    | Analisi della Configurazione dell'Ambiente e Scrutinio del Background . . . . . | 22        |
| 6.3      | Generazione delle Etichette . . . . .                                           | 23        |
| 6.3.1    | Costruzione dell'Interfaccia Olografica . . . . .                               | 23        |
| 6.3.2    | Orientamento Dinamico Verticale . . . . .                                       | 23        |
| 6.3.3    | Gestione dell'Affollamento e Impilamento Verticale . . . . .                    | 23        |
| 6.3.4    | Inseguimento di Oggetti Manipolati . . . . .                                    | 24        |
| 6.3.5    | Culling Tronco-Conico e Regolazione del Sovraffollamento . . . . .              | 24        |
| 6.4      | Persistenza Spaziale: Meta Spatial Anchors . . . . .                            | 24        |
| 6.4.1    | Architettura del Ciclo di Vita delle Ancore . . . . .                           | 24        |
| <b>7</b> | <b>Strumenti per l'Editor Unity e Automazione del Setup</b>                     | <b>25</b> |
| 7.1      | Inquadramento e Motivazione: Riduzione del Lavoro Manuale . . . . .             | 25        |
| 7.2      | Generazione Automatica e Validazione della Scena . . . . .                      | 25        |
| 7.2.1    | Pipeline di Creazione Deterministica della Scena . . . . .                      | 25        |
| 7.3      | Automazione del Manifest Android e Build Pipeline . . . . .                     | 26        |
| 7.3.1    | Risoluzione del Traffico in Chiaro LAN . . . . .                                | 26        |

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>8</b> | <b>Valutazione e Risultati</b>                                                   | <b>27</b> |
| 8.1      | Analisi Qualitativa nelle Diverse Modalità Operative . . . . .                   | 27        |
| 8.1.1    | Modalità Passthrough su Visore Reale (Meta Quest 3) . . . . .                    | 27        |
| 8.1.2    | Modalità Video Offline . . . . .                                                 | 28        |
| 8.1.3    | Modalità VR Sintetica (Ambiente 3D Simulato) . . . . .                           | 28        |
| 8.2      | Profilazione Quantitativa delle Latenze e Strategie di Streaming . . . . .       | 30        |
| 8.2.1    | Studio Comparativo A/B sul Prompt: Prefill vs Decodifica . . . . .               | 30        |
| 8.3      | Validazione Empirica della Re-identificazione: Test a Doppio Passaggio . . . . . | 31        |
| 8.4      | Limiti dello Scene Understanding Nativo e Ruolo della Global Mesh . . . . .      | 31        |
| <b>9</b> | <b>Conclusioni e Sviluppi Futuri</b>                                             | <b>33</b> |
| 9.1      | Sintesi dei Risultati Conseguiti . . . . .                                       | 33        |
| 9.2      | Problematiche Rilevate: Il Conflitto Architetture con Vuforia . . . . .          | 33        |
| 9.3      | Sviluppi Futuri . . . . .                                                        | 34        |
| 9.3.1    | Inferenza On-Device tramite Modelli Quantizzati (Edge AI) . . . . .              | 34        |
| 9.3.2    | Sessioni Collaborative Multi-Utente su Cloud Anchors . . . . .                   | 34        |
| 9.3.3    | Interazione Vocale Multimodale Bidirezionale (VUI) . . . . .                     | 34        |
| 9.3.4    | Inseguimento Dinamico 6-DoF di Oggetti Manipolati . . . . .                      | 35        |

# 1. Introduzione

Il presente progetto, denominato **Document Your Reality** ha l'obiettivo di progettare e implementare un sistema in grado di osservare l'ambiente in tempo reale, identificare entità rilevanti e generare automaticamente documentazione contestualizzata persistente. Tale documentazione assume la forma di annotazioni ed etichette virtuali ancorate direttamente agli oggetti fisici reali, consentendo a un operatore subentrante di consultare visivamente, comprendere e riutilizzare lo storico lasciato dal predecessore.

## 1.1 Obiettivi e Task di Progetto

Nel seguente progetto sono stati svolti tre principali task:

1. **Riconoscimento tramite MLLM:** Acquisizione del flusso visivo dell'ambiente e rilevamento automatico delle entità presenti nella scena.
2. **Realizzazione della Documentazione:** Generazione delle osservazioni dotate di persistenza.
3. **Interfaccia e Ancoraggio Spaziale in Mixed Reality:** Progettazione di un'interfaccia utente fruibile in Mixed Reality su visore e simulatore Unity.

## 1.2 Flessibilità di Deployment e Ambito di Esecuzione

L'applicativo Unity è sviluppato attraverso una pipeline unificata in grado di adattarsi dinamicamente all'hardware a disposizione:

- **Modalità Passthrough (Headset reale):** Sfrutta l'accesso alle telecamere del visore (*Passthrough Camera*) per l'inferenza MLLM.
- **Modalità Video:** Consente il collaudo e l'analisi online a partire da file multimediali .mp4, sovrapponendo dinamicamente le annotazioni sull'interfaccia 2D dello schermo senza necessitare del visore collegato.
- **Modalità VR (Stanza Virtuale):** Ambiente 3D simulato per test interattivi, in cui le etichette vengono posizionate sugli oggetti virtuali mantenendo la medesima logica di ancoraggio.

L'intera architettura è stata pensata per ridurre al minimo il lavoro manuale di setup per l'utente.

## 1.3 Struttura del Documento

La presente relazione è organizzata nei seguenti capitoli:

- Il **Capitolo 2 (Architettura del Sistema)** descrive la topologia distribuita client-server e lo stack tecnologico adottato.
- Il **Capitolo 3 (Sviluppo del Backend: Intelligenza Artificiale e Logica di Dominio)** descrive l'integrazione del modello MLLM, la formattazione dei prompt e il processo per la re-identificazione.
- Il **Capitolo 4 (Gestione della Memoria e dello Storage)** descrive il sistema di archiviazione locale (*MemoryStore*), la separazione dei contesti, il processo di distillazione ricorsiva della memoria semantica e le politiche di limitazione del budget di caratteri.
- Il **Capitolo 5 (Sviluppo del Frontend Unity: Gestione Frame e Interfaccia)** analizza la gestione delle sorgenti visive (Passthrough, VR, Video), l'algoritmo di valutazione della nitidezza, l'interfaccia HUD e la normalizzazione degli input (controller e hand-tracking).
- Il **Capitolo 6 (Spatial Computing e Posizionamento AR)** illustra la trasduzione dai bounding box 2D alle coordinate 3D, l'ancoraggio su mesh MRUK, le tecniche anti-affollamento e la persistenza 6-DoF mediante Meta Spatial Anchors.
- Il **Capitolo 7 (Strumenti per l'Editor Unity e Automazione del Setup)** descrive gli strumenti sviluppati per ridurre il lavoro manuale di configurazione.
- Il **Capitolo 8 (Valutazione e Risultati)** presenta i risultati sperimentali ottenuti e analisi delle latenze.
- Il **Capitolo 9 (Conclusioni e Sviluppi Futuri)** riassume gli obiettivi conseguiti, discute i vincoli emersi e delinea le prospettive future di sviluppo.

## 2. Architettura del Sistema

Il seguente capitolo si pone l'obiettivo di descrivere l'architettura del sistema. In particolare, descrive la topologia distribuita client-server e lo stack tecnologico adottato.

### 2.1 Topologia Client-Server e Pipeline di Scansione

Il sistema adotta un'architettura distribuita client-server disaccoppiata, progettata per separare i compiti di rendering, tracciamento spaziale e interazione in tempo reale (*Edge/Client*) dal carico computazionale richiesto dall'inferenza dei modelli multimodali (*Backend/Inference Server*).

A causa dei vincoli di calcolo e di consumo energetico imposti dai SoC (System on a Chip) integrati nel visore, l'esecuzione locale di un modello MLLM per compiti di grounding visivo risulterebbe impraticabile. Di conseguenza, il visore opera come client, delegando l'elaborazione semantica a un server locale mediante una rete wireless (LAN).

La pipeline sviluppata in tale progetto si articola nelle seguenti fasi sequenziali e asincrone:

1. **Cattura e Pre-elaborazione del Frame:** Il client Unity estrae il frame sorgente (proveniente da Passthrough Camera, telecamera virtuale o flusso video) e lo codifica in formato compresso JPEG, con una qualità pari a 85. Il ridimensionamento e la ricodifica destinati al modello sono demandati al backend (Sezione 3.2.1).
2. **Inoltro della Richiesta HTTP:** Il client prepara una richiesta HTTP multipart (*WWWForm*) indirizzata all'endpoint:

POST /scan/stream (2.1)

Il corpo della richiesta include il payload binario dell'immagine, l'identificativo dell'ambiente (*room\_id*) e la sorgente di esecuzione (*source*: passthrough, VR o video).

3. **Elaborazione Backend e Query MLLM:** Il backend Python (FastAPI) acquisisce il frame e prepara il prompt contestuale. La richiesta viene arricchita con la lista dei riferimenti semantici degli oggetti già censiti, contingentata entro un budget di caratteri configurabile (*scan.known\_objects\_char\_budget*, impostato a 9000) e fino a un numero massimo di entità note contemporanee (*scan.max\_known\_objects\_in\_prompt*, impostato a 40). L'inferenza viene inoltrata verso il server di modelli conforme all'interfaccia OpenAI (es. Ollama o vLLM).
4. **Streaming Progressivo in Formato NDJSON:** Data la natura generativa sequenziale dei modelli linguistici, attendere il completamento dell'intero output comporterebbe una latenza elevata per gli utenti. Il backend trasmette quindi i risultati parziali man mano che i token vengono generati, sfruttando il protocollo NDJSON (*Newline Delimited JSON*). Ogni riga inviata dal server corrisponde a un messaggio serializzato dotato di marcatore di tipo:

- {"t": "label", "label": {...}}: notifica l'identificazione e la delimitazione 2D di un'entità con bounding box normalizzato [0, 1000];
  - {"t": "error", "error": "..."}: segnala un'anomalia di inferenza generata dal modello;
  - {"t": "done", "done": {...}}: notifica la conclusione dello stream del frame corrente.
5. **Ricezione Asincrona e Posizionamento AR:** Sul client, il flusso UTF-8 si decodifica riga per riga mediante buffer. Non appena viene intercettato un blocco *label*, viene istanziata progressivamente l'interfaccia nel punto tridimensionale calcolato per raycasting, riducendo la latenza percepita.
  6. **Vincolo di Persistenza Geometrica (*Anchor Binding*):** Subito dopo aver stabilito la posizione 3D del marker nello spazio del visore, il client invia una richiesta asincrona di associazione all'endpoint:

POST /objects/{entity\_id}/anchor (2.2)

Il payload trasmette al database del backend l'identificativo UUID dell'ancora spaziale generata dal runtime Meta XR (*anchor\_uuid*), garantendo la ricostruzione dello stato della stanza nelle sessioni future.

Poiché il processo di analisi visiva su modelli MLLM di grandi dimensioni richiede tempi prolungati, il timeout della scansione sul client fissato a 420s è configurato per essere superiore al timeout interno del backend pari a 360s. L'intera pipeline che descrive il ciclo di vita del nostro sistema è illustrata schematicamente in Figura 2.1.

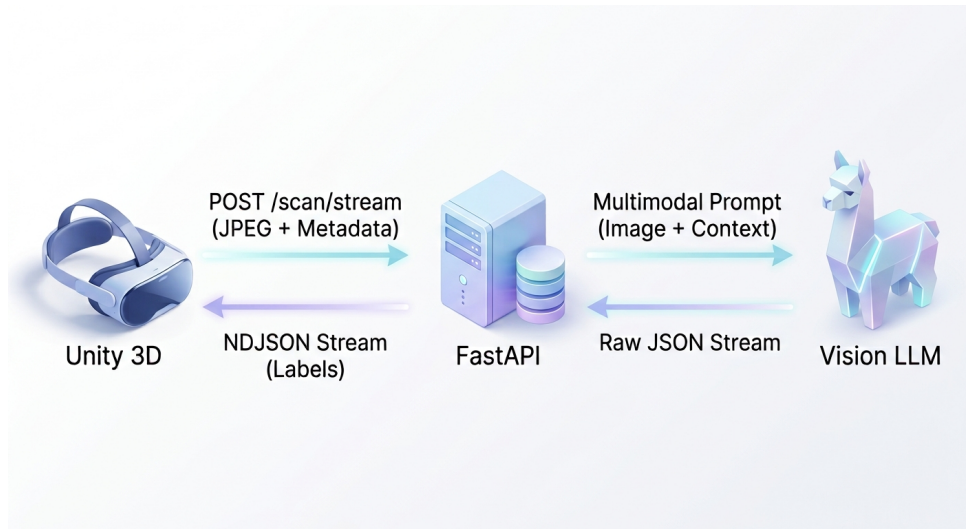


Figura 2.1: Pipeline del ciclo di vita del sistema.

## 2.2 Sistema di Discovery Dinamico su UDP

All'interno della APK per Android/Meta Quest, l'URL predefinito configurato nell'*Inspector* di Unity punta tipicamente all'indirizzo `http://127.0.0.1:8000`. Tuttavia, se eseguito all'interno del sistema operativo del visore, `127.0.0.1` risolve localmente sull'headset stesso e non sulla workstation che ospita il backend, causando un fallimento immediato delle comunicazioni.

Il sistema implementa un meccanismo di autoconfigurazione (*Zero-Configuration Discovery*) basato su broadcast UDP.

### 2.2.1 Protocollo di Handshake e Risoluzione del Peer

Il meccanismo di discovery opera sulla porta UDP 8001:

1. All'avvio dell'applicazione Unity, prima di instradare qualsiasi richiesta HTTP, il client verifica la validità dell'indirizzo.
2. Il client avvia un thread worker dedicato in background, apre un'istanza di `UdpClient` e trasmette il datagramma ASCII:

$$\mathcal{P}_{\text{probe}} = \text{"DYR?"} \longrightarrow 255.255.255.255:8001 \quad (2.3)$$

3. La server, un demon asincrono si pone in ascolto continuo sulla porta 8001 su tutte le interfacce disponibili.

### 2.2.2 Instradamento Multi-Interfaccia e Kernel Routing

Una criticità architetturale riguarda le macchine di sviluppo provviste di interfacce di rete multiple (es. adattatore Wi-Fi locale, Ethernet e VPN/Overlay Network quali Tailscale). Un'assegnazione statica dell'IP da parte del backend rischierebbe di restituire l'indirizzo della VPN, irraggiungibile per il visore collegato unicamente alla rete Wi-Fi locale.

Per risolvere tale ambiguità, viene determinato l'indirizzo IP locale corretto per ogni singolo probe ricevuto, interrogando la tabella di routing del kernel del sistema operativo ospitante. La chiamata di sistema *connect* su un socket UDP interroga il kernel per individuare quale interfaccia di rete possiede la rotta verso il mittente del probe (*peer*). Recuperando il rispettivo indirizzo, il responder serializza la risposta:

$$\mathcal{R}_{\text{reply}} = \text{"DYR http://"} + \text{local\_ip} + \text{" "} + \text{port} \quad (2.4)$$

Il client Unity riceve il pacchetto, ne isola il prefisso di controllo ("DYR ") e sovrascrive a runtime la proprietà *backendBaseUrl* con l'URI validato.

## 2.3 Stack Tecnologico

L'intero ecosistema software si basa su tecnologie open-source e framework moderni, garantendo aderenza agli standard e riproducibilità sperimentale.

### 2.3.1 Client e Frontend Immersivo (Unity Engine)

Il motore di visualizzazione e tracking è sviluppato all'interno di Unity, sfruttando:

- **Meta XR SDK:** Fornisce le astrazioni necessarie per la gestione del passthrough, il caricamento della mesh dell'ambiente e l'interrogazione geometrica delle superfici mediante raycasting su collider reali.
- **Meta Spatial Anchor API:** Meccanismo di tracciamento 6-DoF per l'ancoraggio persistente delle annotazioni, con salvataggio delle ancore.
- **UnityWebRequest & DownloadHandlerScript:** Gestione asincrona ad alte prestazioni del networking basato su coroutine, con deserializzazione streaming riga-per-riga dei payload JSON tramite la libreria nativa *JsonUtility*.

### 2.3.2 Backend Computazionale e Middleware

Il backend applicativo è implementato in Python 3.11+ e si struttura attorno alle seguenti librerie:

- **FastAPI (v0.115.0+)**: Framework web asincrono impiegato per esporre le API RESTful e le risposte in streaming via HTTP/1.1;
- **Uvicorn (v0.30.0+)**: Per la gestione della concorrenza su event-loop;
- **HTTPX (v0.27.0+)**: Client HTTP asincrono per la propagazione delle richieste verso i server di inferenza MLLM locali o remoti;
- **Pydantic (v2.8.0+)**: Validazione dichiarativa dei dati serializzati tra Unity e Python.
- **Pillow (v10.4.0+) & imageio-ffmpeg (v0.6.0+)**: Manipolazione delle bitmap in memoria, ricampionamento geometrico per il parsing delle immagini e ingestione di sequenze video compresse;
- **Jinja2 (v3.1.0+)**: Motore di templating adottato per l'assemblaggio dinamico e modulare dei prompt di sistema destinati al modello di visione.

### 2.3.3 Motore di Visione e Grounding Multimodale (MLLM)

Il riconoscimento delle entità è affidato a Vision-Language Models.

L'architettura supporta modelli come la famiglia *Gemma 4:e2b* o architetture più potenti come *Qwen 2.5/3.6 VL*, purché capaci di restituire bounding box. La configurazione consente la parametrizzazione precisa della temperatura di generazione (*temperature: 0* per garantire il massimo determinismo analitico) e la limitazione del numero massimo di token emessi (*scan\_max\_tokens: 2048*). Per i nostri esperimenti sono stati utilizzati entrambi i modelli citati in precedenza.

## 3. Sviluppo del Backend: Intelligenza Artificiale e Logica di Dominio

Tale capitolo descrive l'integrazione del modello MLLM, la formattazione dei prompt e il processo per la re-identificazione.

### 3.1 API e Gestione delle Richieste

Il backend applicativo è progettato come un servizio asincrono basato su FastAPI, concepito per orchestrare l'interazione tra i flussi visivi inviati dai dispositivi XR, il modello di visione-linguaggio (MLLM) e lo strato di persistenza semantica locale (*MemoryStore*).

#### 3.1.1 Architettura degli Endpoint REST

Le interfacce esposte dal backend si dividono in endpoint di sincronizzazione dello stato, diagnostica e pipeline di scansione visiva:

- **GET /health e GET /llm/health:** Restituiscono lo stato del backend, il numero di entità e annotazioni caricate in memoria e lo stato di prontezza del modello linguistico (*probe\_state: untested, up, down*).
- **GET /objects:** Chiamato dal visore al bootstrap per ripristinare le etichette persistenti. Si applica un partizionamento logico degli ambienti, impedendo a una sessione virtuale di ereditare annotazioni generate nell'ambiente reale.
- **POST /objects/{entity\_id}/anchor:** Vincola l'UUID dell'ancora spaziale generata dal runtime Meta XR all'entità censita, aggiornandone i metadati.
- **POST /scan/stream:** Costituisce il canale principale di inferenza visiva. Riceve un payload multipart codificato con il frame sorgente (*image*), la sorgente (*source*) e l'identificativo opzionale della stanza (*room\_id*).

#### 3.1.2 Concorrenza e Isolamento delle Risorse

I modelli MLLM impongono un vincolo sull'occupazione di memoria VRAM e sull'accesso sequenziale ai contesti di calcolo. La sovrapposizione di chiamate di visione concorrenti da parte di molteplici frame porterebbe a un degrado prestazionale della workstation ospitante.

Per prevenire l'accodamento di frame obsoleti durante i movimenti dell'operatore, il backend impone un blocco mediante un semaforo asincrono globale.

Se il modello è occupato nell'elaborazione del frame precedente, la richiesta entrante viene immediatamente rigettata con stato *ok: false*. In questo modo il client Unity evita l'attesa di risultati riferiti a posizioni visuali non più allineate con il campo visivo attuale dell'operatore.

### 3.1.3 Elaborazione in Background e Gestione Errori

La distillazione della memoria semantica a lungo termine richiede una seconda interrogazione al modello linguistico. Per non penalizzare la reattività dello streaming verso il visore, questa operazione viene sganciata dal ciclo principale ed eseguita asincronamente in background.

Il middleware di logging HTTP monitora la latenza end-to-end di ogni richiesta, intercettando gli errori di validazione del payload. Eventuali fallimenti o timeout dell'infrastruttura di inferenza vengono intercettati come *LlmError* e mappati sullo stato HTTP 502 (*Bad Gateway*).

## 3.2 Integrazione con il Modello MLLM

Le chiamate vengono incapsulate verso server di inferenza compatibili con lo standard OpenAI Chat Completions, supportando motori open-source quali Ollama e vLLM.

### 3.2.1 Normalizzazione Ottica e Pre-elaborazione del Frame

I flussi d'immagine catturati da telecamere ad alta definizione comportano payload eccessivi e generano un numero elevato di token visivi, degradando i tempi di elaborazione. Il frame binario JPEG viene elaborato tramite la libreria Pillow imponendo un vincolo sul lato minore e la ricodifica JPEG a qualità 80. L'algoritmo non applica mai upscaling su immagini di risoluzione inferiore. I byte risultanti vengono serializzati in formato Base64 all'interno del payload data-URI.

### 3.2.2 Parsing Incrementale dello Stream JSON

I server di inferenza inviano token testuali continui formattati secondo il protocollo *Server-Sent Events* (SSE), contraddistinti dal prefisso *data:*. La struttura attesa in output è un array JSON racchiuso all'interno di un oggetto: `{"objects": [ {...}, {...} ]}`.

Attendere la chiusura dell'array o il token finale *[DONE]* impedirebbe il funzionamento in streaming. Per estrarre gli oggetti non appena completati viene analizzato il buffer di caratteri in ingresso, vengono gestiti i caratteri di escape.

### 3.2.3 Terminazione Anticipata dello Stream

Sebbene il prompt imponga un tetto massimo di rilevamenti, i modelli linguistici tendono a produrre descrizioni oltre i vincoli definiti. Troncare la risposta a valle dell'intera generazione lascerebbe invariato il tempo di attesa dell'utente.

Per convertire il limite di rilevamento in un effettivo risparmio temporale, il sistema tronca forzatamente il generatore asincrono. In particolare, la connessione HTTP sottostante viene chiusa, interrompendo immediatamente l'esecuzione sulla GPU. Nei test condotti su una scena di prova, questa soluzione dimezza la latenza da 11.3 a 5.8 secondi.

### 3.3 Prompt Engineering e Regole di Estrazione

La conversione di un'immagine non strutturata in etichette spaziali stabili è governata da un prompt di sistema compilato tramite Jinja2. L'architettura evita pipeline a chiamate multiple (rilevamento e successiva descrizione), condensando il compito in un'unica inferenza.

#### 3.3.1 Dicotomia Operativa: Shape A e Shape B

Quando il backend dispone già di entità memorizzate nella scena, la sezione iniziale del prompt inietta la lista degli oggetti noti nel formato sintetico:

```
id | name | how to tell this exact one apart | what is known about it
```

Il modello è istruito a emettere ciascun oggetto conformemente a due schemi mutuamente esclusivi (Figura 3.1):

| Shape A: Riconoscimento (Entità Nota)                                                                                                                                                                                                                   | Shape B: Documentazione (Nuova Entità)                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Adottata quando l'oggetto corrisponde a un elemento della <i>cue list</i> . Omette descrizioni ridondanti.<br><br><pre>{<br/>  "known_id": "obj_3f2a91",<br/>  "x0": 386, "y0": 495,<br/>  "x1": 823, "y1": 756,<br/>  "is_new_info": false<br/>}</pre> | Adottata per oggetti mai rilevati in precedenza. Richiede la documentazione completa.<br><br><pre>{<br/>  "known_id": "",<br/>  "name": "grey storage bin",<br/>  "distinctive": "grey bin near desk",<br/>  "x0": 386, "y0": 495,<br/>  "x1": 823, "y1": 756,<br/>  "title": "Storage bin",<br/>  "body": "Open bin with tools.",<br/>  "steps": [],<br/>  "tags": ["storage", "tools"],<br/>  "is_new_info": true<br/>}</pre> |

Figura 3.1: Confronto strutturale tra i payload Shape A (Re-identificazione) e Shape B (Prima Osservazione).

L'adozione di Shape A abbate il consumo di token per entità note da circa 120 a meno di 25, minimizzando il rischio che il modello riformuli descrizioni di oggetti statici. Il flag *is\_new\_info* viene attivato su Shape A solo in caso di alterazioni fisiche evidenti.

#### 3.3.2 Grounding Spaziale con Vertici Nominati e Prevenzione delle Trasposizioni

Il sistema richiede la delimitazione degli oggetti mediante coordinate intere normalizzate nel range  $[0, 1000]$ .

Una scelta architetturale risiede nell'imposizione di chiavi esplicite e separate per ciascun asse ("*x0*", "*y0*", "*x1*", "*y1*"), vietando l'uso di liste anonime di quattro numeri (es.  $[y0, x0, y1, x1]$ ). Modelli di famiglie distinte seguono convenzioni coordinate discordanti (Gemma adotta l'ordinamento  $[y_{\min}, x_{\min}, y_{\max}, x_{\max}]$ , mentre Qwen adotta  $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ ). L'interpretazione errata di una tupla determinerebbe la trasposizione degli assi cartesiani, traslando le etichette su oggetti errati pur mantenendo una struttura JSON formalmente valida.

#### 3.3.3 Verifica Empirica tramite Pattern Anti-Diagonale

Per validare l'allineamento spaziale del modello prima della messa in servizio viene generata a runtime un'immagine di test sintetica con due figure geometriche collocate sulla diagonale secondaria (*anti-diagonale*):

- Quadrato rosso posizionato in alto a destra:  $x \in [400, 600], y \in [40, 200]$ ;
- Cerchio blu posizionato in basso a sinistra:  $x \in [40, 240], y \in [280, 440]$ .

Tale immagine di test è riportata in Figura 3.2.

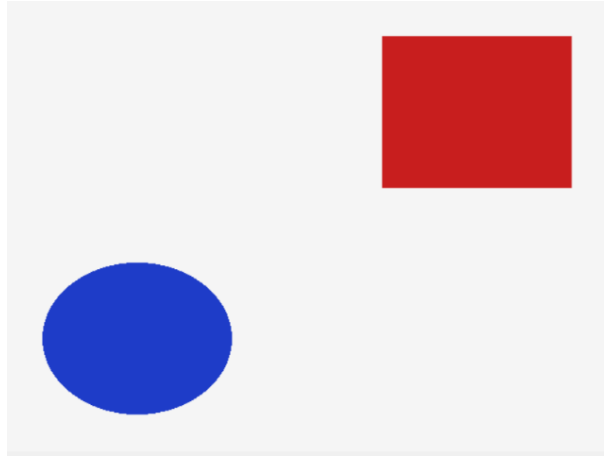


Figura 3.2: Immagine di test

Sull’anti-diagonale, uno scambio tra ascisse e ordinate proietta il quadrato rosso in basso a sinistra e il cerchio blu in alto a destra. Lo script rileva tale anomalia mediante il calcolo del centro geometrico normalizzato  $(c_x, c_y)$  e ne segnala l’inversione.

### 3.3.4 Prioritizzazione e Conservazione delle Nuove Informazioni

Attraverso una regola del prompt di scansione viene imposto al modello un ordinamento:

*”If anything in this frame is not on the list, the first entry is one of those [...]. The reason is that they are the only ones that can be lost.”*

Poiché lo stream viene troncato al raggiungimento di *max\_objects\_per\_frame*, se il modello elencasse prioritariamente gli oggetti già noti esaurirebbe i posti disponibili prima di descrivere i nuovi elementi. Le entità note sono già persistite nel database; un’omissione nel frame corrente non ne pregiudica l’esistenza futura. Al contrario, un oggetto nuovo non verbalizzato nei primi slot verrebbe scartato senza essere registrato nel sistema.

## 3.4 Logica di Re-identificazione e Persistenza Semantica

Per elaborare le istanze generate dal modello viene istanziato un meccanismo di risoluzione dell’identità multilivello a tolleranza di errore.

### 3.4.1 Pipeline di Risoluzione dell’Identità

La validità geometrica iniziale viene verificata tramite un predicato che scarta bounding box degeneri con estensione inferiore al 2% su ciascun asse. Successivamente, il sistema tenta la re-identificazione applicando una cascata di tre criteri decrescenti:

#### 1. Livello 1 - Riconoscimento Esplicito da Modello (*known\_id*):

Se il campo *known\_id* è valorizzato, il resolver esegue un match sulla base dell’id. I modelli compatti tendono talvolta a troncare o omettere il prefisso *obj\_*. In caso di troncamento parziale, il candidato viene associato se e solo se esiste una corrispondenza biunivoca non ambigua nel database.

## 2. Livello 2 - Tracciamento Geometrico per Sovrapposizione (*Box Overlap IoU*):

Se il modello non associa il *known\_id* ma rileva un oggetto con denominazione compatibile, il sistema calcola l'*Intersection over Union* (IoU) tra il box corrente  $B_c$  e l'ultimo box registrato  $B_e$  per ogni entità nello stesso ambiente:

$$\text{IoU}(B_c, B_e) = \frac{\text{Area}(B_c \cap B_e)}{\text{Area}(B_c \cup B_e)} \geq \tau_{\text{IoU}} \quad (3.1)$$

con soglia  $\tau_{\text{IoU}} = 0.55$ . Se l'overlap supera la soglia, l'osservazione viene ricondotta all'entità preesistente.

## 3. Livello 3 - Fusione Temporale per Denominazione Recente:

Se l'overlap geometrico non è applicabile, ad esempio, a causa di un rapido cambio di inquadratura o movimento dell'oggetto, viene valutata la vicinanza temporale entro una finestra configurabile, impostata a 120 secondi:

$$\Delta t = t_{\text{now}} - t_{\text{last\_seen}} \leq 120 \text{ s} \quad (3.2)$$

Se un'entità con lo stesso nome normalizzato è stata osservata nell'intervallo  $\Delta t$ , i rilevamenti vengono unificati.

Per evitare collisioni, due rilevamenti distinti appartenenti al medesimo frame non possono mai collassare sulla medesima entità.

### 3.4.2 Filtro delle Annotazioni Ridondanti

Per impedire che riformulazioni puramente stilistiche del testo sovrascrivano l'etichetta associata all'oggetto, vengono imposti i seguenti criteri di filtro:

- **Cooldown Temporale:** Tra due annotazioni successive di stato sulla medesima entità deve intercorrere un intervallo minimo configurabile, impostato a 25 secondi.
- **Differenziale Lessicale (*Bag-of-Words*):** Il sistema calcola l'insieme dei termini significativi del nuovo corpo descrittivo  $W_{\text{new}}$  rispetto al precedente  $W_{\text{prev}}$ :

$$\Delta W = W_{\text{new}} \setminus W_{\text{prev}} \quad (3.3)$$

Se  $\Delta W = \emptyset$ , la variazione non genera una nuova osservazione.

### 3.4.3 Distillazione della Memoria Evolutiva (*EntityMemory*)

Qualora l'osservazione superi i filtri, il sistema invoca *update\_entity\_memory*. Il modello linguistico estrae una sintesi incrementale strutturata, vincolando il numero di asserzioni atomiche e i punti aperti operativi entro i limiti fissati dalla configurazione (*memory.max\_facts*, impostato a 8, e *memory.max\_open\_points*, impostato a 4). In caso di indisponibilità del server MLLM, il modulo esegue un fallback deterministico, preservando la continuità operativa del grafo di conoscenza.

## 4. Gestione della Memoria e dello Storage

Il seguente capitolo descrive il sottosistema di archiviazione locale (*MemoryStore*), la separazione dei contesti, il processo di distillazione ricorsiva della memoria semantica e le politiche di limitazione del budget di caratteri.

### 4.1 Persistenza dei Dati e Struttura dello Storage

Nei sistemi di documentazione in Mixed Reality operanti con modelli linguistici multimodali, la gestione della persistenza costituisce un elemento importante per garantire continuità tra diverse sessioni. Un sistema di memorizzazione deve da un lato garantire l'archiviazione duratura e strutturata dell'intero storico osservato su disco, dall'altro supportare accessi in lettura a latenza quasi nulla ( $O(1)$ ) per non degradare la frequenza di aggiornamento del visore.

Il sottosistema di persistenza del backend è implementato attraverso la classe *MemoryStore*, la quale gestisce la persistenza serializzata su disco formattata in file JSON atomici.

#### 4.1.1 Organizzazione del File System e Indici in Memoria

La cartella radice dei dati (*data/*) viene suddivisa in tre percorsi distinti:

- *entities/*: Contiene le definizioni degli oggetti tracciati, salvate in file json.
- *annotations/*: Contiene lo storico di tutte le osservazioni generate nel tempo dall'MLLM, salvate in file json.
- *captures/*: Archivio delle immagini sorgente.

All'avvio del servizio, il componente scansiona la directory e popola gli indici in RAM, garantendo il recupero immediato dei record durante l'elaborazione dei frame di scansione.

#### 4.1.2 Partizionamento degli Ambienti

L'esecuzione del software sia in ambienti simulati (VR) sia nel contesto reale (visore con Passthrough o flussi video preregistrati), viene consentita tramite una funzione discriminatrice.

La determinazione del dominio garantisce che, interrogando le entità viste di recente o compilando la lista di contesto per l'MLLM, non si verifichino interferenze.

#### 4.1.3 Struttura dei Dati: Entità e Bounding Box

La classe *Entity* incapsula l'identità di un elemento fisico rilevato. Ogni entità è salvata come documento JSON contenente metadati, ancoraggi fisici, coordinate visive normalizzate e lo stato sintetico corrente *EntityMemory* come si può notare nell'esempio riportato di seguito:

```

{
  "entity_id": "obj_lacd0b8f",
  "name": "office chair",
  "distinctive": "black ergonomic chair with armrests",
  "anchor_uuid": "",
  "anchor_kind": "",
  "qr_payload": "",
  "room_id": "",
  "source": "video:room_scan.mp4",
  "last_box": {
    "x0": 0.444,
    "y0": 0.435,
    "x1": 0.591,
    "y1": 0.693
  },
  "created_at": "2026-09-04T15:46:13.248307Z",
  "last_seen_at": "2026-09-04T15:46:13.256633Z",
  "tags": [
    "furniture",
    "seating"
  ],
  "annotation_count": 1,
  "memory": {
    "summary": "",
    "facts": [],
    "open_points": [],
    "updated_at": "2026-09-04T15:46:13.248319Z",
    "version": 0
  }
}

```

Il modello *ObjectBox* normalizza lo spazio bidimensionale delle immagini convertendo le coordinate  $[0, 1000]$  restituite dal modello di visione in un sistema a virgola mobile ristretto nell'intervallo  $[0.0, 1.0]$ .

Viene inoltre imposto un vincolo sull'area minima racchiusa:

$$\text{Area} = (x_{\max} - x_{\min}) \cdot (y_{\max} - y_{\min}) > 0.0004 \quad (4.1)$$

scartando box con estensione inferiore allo 0.04% della superficie totale, interpretate sistematicamente come rumore o falsi positivi dell'estrattore semantico.

#### 4.1.4 Struttura delle Annotazioni e Salvataggio delle Acquisizioni Visive

Mentre l'entità rappresenta l'oggetto fisico persistente, la classe *StoredAnnotation* formalizza l'osservazione generata a valle dell'inferenza. Ciascuna annotazione conserva l'associazione biunivoca verso l'entità di appartenenza (*entity\_id*), il delimitatore visuale *box*, il modello linguistico impiegato (*llm\_model*) e il percorso del frame acquisito (*image\_file*) come riportato nel seguente esempio:

```

{
  "annotation_id": "ann_0d07c9812276",
  "entity_id": "obj_e209f1da",
  "room_id": "fa78158e-0fe2-9acc-537b-579a6b12e232",
  "status": "draft",
  "created_at": "2026-09-04T15:44:10.784834Z",
  "image_file": "C:\\...\\data\\captures\\ann_0d07c9812276.jpg",
  "box": {
    "x0": 0.543, "y0": 0.725, "x1": 0.956, "y1": 1.0
  },
  "llm_model": "Qwen/Qwen3.6-35B-A3B-FP8",
  "annotation": {
    "object_name": "metal chair",
    "title": "Metal Chair",
    "body": "Polished metal chair in the foreground.",
    "steps": [],
    "tags": ["furniture", "chair", "metal"]
  }
}

```

Per quanto riguarda il salvataggio del frame il flusso binario viene deserializzato in memoria tramite Pillow, forzato nello spazio colore standard sRGB e codificato in formato compresso JPEG con fattore di qualità calibrato a 92.

Qualora un'annotazione o la rispettiva entità genitore vengano eliminate dall'utente, il framework esegue l'eliminazione a cascata, cancellando dal disco il file di definizione JSON.

## 4.2 Distillazione della Memoria Semantica (Memory System)

Negli scenari d'uso prolungati, l'accumulo sequenziale di osservazioni genera rapidamente un volume elevato di descrizioni. Inviare l'intera cronologia testuale di un oggetto all'interno dei prompt di scansione successivi comporterebbe la saturazione della finestra di contesto del modello linguistico.

Per risolvere tale criticità, il sistema integra un motore di distillazione asincrona basato sul template di prompt *memory.j2*, con lo scopo di comprimere lo stato dell'oggetto in una struttura informativa compatta (*EntityMemory*).

### 4.2.1 Il Ciclo di Sintesi Incrementale

Nel momento in cui avviene il salvataggio di una nuova annotazione, il backend emette un task asincrono in background e il template Jinja2 *memory.j2* struttura la richiesta fornendo al modello linguistico tre elementi concettuali:

1. **Identità dell'Oggetto**
2. **Stato Progresso della Memoria**
3. **Nuova Osservazione**

Le regole di distillazione vincolano le informazioni estratte dal modello linguistico:

- **Stato Complessivo:** Massimo 50 parole. Descrive la condizione operativa corrente dell'oggetto. Vige la regola della precedenza cronologica, garantendo che variazioni fisiche evidenti sovrascrivano asserzioni storiche non più attuali;
- **Fatti Atomici Durevoli:** Limitati alla quota massima determinata da configurazione, impostata a 8 con una lunghezza massima di 12 parole per fatto.
- **Punti Aperti Operativi:** Limitati alla quota fissata a 4.

Il modello è istruito a generare come risposta esclusivamente un dizionario JSON, strutturato secondo la grammatica:

$$\mathcal{S}_{\text{memory}} = \{\text{"summary": str, "facts": list[str], "open_points": list[str]}\} \quad (4.2)$$

In caso di mancata risposta o errore del backend di inferenza, il sistema invoca un fallback, che appende la nuova descrizione e incrementa il contatore di versione senza corrompere la consistenza dello stato progressivo.

### 4.2.2 Grafo di Relazione Semantica e Ordinamento per Sezioni

Il sistema ricostruisce le relazioni intercorrenti tra gli elementi della scena. L'affinità semantica tra l'oggetto target  $e_t$  e un generico elemento  $e_i \in \mathcal{E}$  censito nel medesimo ambiente è quantificata mediante una funzione pesata:

$$\text{Score}(e_t, e_i) = 2.0 \cdot |\text{Tags}(e_t) \cap \text{Tags}(e_i)| + 1.0 \cdot \mathbb{I}(\text{room\_id}(e_t) = \text{room\_id}(e_i)) \quad (4.3)$$

Gli oggetti che condividono parole chiave e contengono annotazioni pregresse valide vengono inseriti in ordine decrescente di punteggio all'interno di una struttura, fino a un numero massimo di entità configurabile, impostato a 3. Tale grafo non è persistito su disco: viene ricalcolato a runtime ogni volta che il contesto completo di un oggetto viene richiesto.

Nel generare il blocco descrittivo finale per l'interfaccia o per prompt dettagliati, il sistema alloca un tetto massimo di memoria testuale configurabile, impostato a 1800 caratteri.

## 5. Sviluppo del Frontend Unity: Gestione Frame e Interfaccia

Il seguente capitolo analizza la gestione delle sorgenti visive (Passthrough, VR, Video), l'algoritmo di valutazione della nitidezza, l'interfaccia HUD e la normalizzazione degli input (controller e hand-tracking).

### 5.1 Architettura delle Sorgenti dei Frame (Frame Sources)

La componente client del sistema, sviluppata all'interno del motore grafico Unity con Universal Render Pipeline (URP), è concepita per operare in modo agnostico rispetto all'origine del segnale visivo

#### 5.1.1 Astrazione e Modello Pinhole

La pipeline di ingestione è strutturata attorno a un'interfaccia:

- **IDyrFrameViewSource**: Espone la struttura, che memorizza la posizione e orientamento della fotocamera e i parametri congelati nell'istante di scatto dell'otturatore;
- **IDyrCapturedRayProvider**: Consente la proiezione di un punto dello spazio immagine normalizzato  $(u, v) \in [0, 1]^2$  in un raggio tridimensionale geometricamente calibrato (*optical raycasting*), tenendo conto delle distorsioni ottiche e delle pose dei sensori.

Quando un modello di visione restituisce le coordinate di un bounding box normalizzato, il centroide bidimensionale viene convertito in un raggio proiettivo nello spazio mondo attraverso la formulazione pinhole.

Tale formulazione garantisce che, anche qualora l'inferenza del modello linguistico richieda diversi secondi durante i quali l'utente ha mosso il capo, il raggio di collocamento AR venga tracciato a partire dal punto di vista esatto in cui l'immagine è stata acquisita, preservando la coerenza prospettica.

#### 5.1.2 Modalità Passthrough e Meta Passthrough Camera API

Alcune modalità sfruttano le funzionalità avanzate dell'SDK Meta XR su architettura Horizon OS. A differenza delle convenzionali applicazioni Mixed Reality in cui il feed video viene renderizzato esclusivamente sul piano di sfondo dello shader, il modulo richiede l'accesso diretto ai sensori RGB.

**Permessi e Ciclo di Ripristino** L'accesso ai dati grezzi delle telecamere esterne è vincolato al permesso Android `horizonos.permission.HEADSET_CAMERA`.

Qualora il sensore primario (sinistro) risulti occupato o bloccato, il sistema alterna dinamicamente l'interrogazione sulla telecamera destra per un massimo di 6 tentativi sequenziali. Inoltre, il sottosistema monitora eventuali

interruzioni provocate dalla sospensione del visore, riattivando la pipeline video non appena il dispositivo torna nello stato di operatività attiva.

**Lettura dei Pixel e Gestione della Desincronizzazione** Un problema tecnico emerso durante la profilazione su Quest 3 risiede nell'asincronia tra il buffer nativo dei colori e la texture GPU associata, per la quale, in determinate finestre temporali, il buffer CPU può presentarsi non dimensionato o privo di pixel validi mentre la texture video è già presente nel rasterizzatore.

Per azzerare i frame scartati, viene implementata una tecnica di fallback su texture off-screen in cui i dati grafici vengono copiati mediante blit accelerato su una RenderTexture di staging in formato *ARGB32* e trasferiti alla memoria di sistema, garantendo l'estrazione continua del frame anche a fronte di latenze.

### 5.1.3 Modalità Sintetica VR e Modalità Video Offline

Per consentire lo sviluppo e la validazione delle funzionalità in assenza del dispositivo fisico l'architettura supporta due sorgenti alternative:

**Modalità VR Sintetica** Rappresenta una simulazione end-to-end dell'ambiente. Viene preparata dinamicamente una stanza virtuale, calcolando il volume complessivo degli arredi e generando pareti e pavimento.

Per evitare che l'infrastruttura di raycasting fallisca intercettando mesh volumetriche vuote o pareti esterne, la procedura applica proceduralmente collider fisici (*BoxCollider*) esclusivamente sui singoli elementi. La cattura dell'inquadratura per l'MLLM è delegata a una telecamera virtuale secondaria (*DyrVrCaptureCamera*) che introduce due vincoli architettureali:

1. **Culling dei Livelli UI** (`labelLayer = 5`): Esclude sistematicamente gli strati grafici delle etichette AR e dell'HUD dal cono visuale dell'MLLM. In questo modo il modello osserva l'ambiente senza incorrere in fenomeni di auto-osservazione (in cui l'IA tenterebbe di descrivere e ancorare le sue stesse etichette proiettate a schermo);
2. **Normalizzazione Prospettica**: Vincola il cono ottico a un campo visivo di  $60^\circ$  verticale con proporzione standard 4 : 3, sostituendo l'ottica ultragrandangolare nativa del visore ( $121^\circ$ ). Tale accorgimento elimina le distorsioni sferiche marginali che degradano pesantemente le capacità di localizzazione spaziale dei modelli di visione.

**Modalità Video** Consente l'ingestione di flussi sequenziali codificati in formato H.264/MP4.

Al raggiungimento del termine del file video, il componente congela l'ultimo frame valido all'interno di una superficie persistente dedicata. L'operatore può quindi continuare a interrogare liberamente il backend sul frame conclusivo senza che la sequenza collassi sul fotogramma iniziale.

### 5.1.4 Analisi della Nitidezza

I movimenti rapidi della testa dell'operatore introducono motion blur.

Per scongiurare tale problematica, viene aperta una finestra temporale di osservazione di 0.5 secondi durante la quale viene analizzato il gradiente ad alta frequenza del segnale visivo mediante un algoritmo che non esegue il calcolo sulla texture ad alta risoluzione, bensì adotta un'approssimazione spaziale discreta:

1. Il frame sorgente viene sottocampionato mediante un passaggio hardware su un buffer transitorio di dimensione fissa  $48 \times 48$  pixel;

2. La matrice dei pixel viene letta nel formato leggero *RGB24*;
3. Viene calcolata la derivata prima discreta mediante differenze finite lungo le direzioni cartesiane limitatamente al canale verde (*G*). La scelta del canale verde è motivata dalla sensibilità dell'occhio umano, che concentra la massima definizione e il miglior rapporto segnale/rumore su tale lunghezza d'onda.

Durante la finestra temporale, il frame viene ri-codificato e memorizzato come candidato alla trasmissione solo se la sua nitidezza supera di almeno il 10% il valore del miglior candidato registrato fino a quel momento ( $\text{score} > 1.1 \cdot \text{best}$ ). Tale euristica garantisce l'invio al backend del frame qualitativamente più risoluto e contrastato.

## 5.2 HUD Immersivo e Sistema di Diagnostica

La fruizione di un'applicazione in Mixed Reality su headset autonomo impone requisiti stringenti di ergonomia visuale. L'utente deve poter verificare costantemente lo stato dei servizi distribuiti, l'esito del riconoscimento e la latenza dei modelli senza che l'interfaccia olografica occluda la visibilità degli oggetti fisici da analizzare.

### 5.2.1 HUD Spaziale Flottante

Nelle modalità immersive (Passthrough e VR) viene generato un pannello spaziale in coordinate mondo, configurato sul layer UI (*layer* = 5) e quindi invisibile alle telecamere di scansione dell'ambiente. In modalità Video lo stato è riportato da un pannello diagnostico bidimensionale sovrapposto allo schermo.

**Posizionamento Ergonomico** L'HUD non viene vincolato rigidamente allo schermo del visore, dinamica nota per indurre cinetosi e affaticamento visivo. Al contrario, il pannello fluttua di fronte allo sguardo dell'operatore, mantenendosi costantemente al di sotto dell'asse ottico primario:

$$\mathbf{p}_{\text{target}} = \mathbf{p}_{\text{head}} + d_{\text{front}} \cdot \mathbf{f}_{\text{head}} - d_{\text{drop}} \cdot \mathbf{u}_{\text{head}} \quad (5.1)$$

dove  $d_{\text{front}} = 1.1$  metri e l'abbassamento è tarato a  $d_{\text{drop}} = 0.38$  metri. Tale sfasamento lascia completamente sgombra la porzione centrale del campo visivo.

## 5.3 Unificazione e Normalizzazione degli Input

La presenza di molteplici modalità operative impone la gestione eterogenea dei dispositivi di puntamento e selezione. L'operatore può interagire con il sistema mediante controller fisici Meta Quest Touch, hand-tracking gestuale nativo senza controller, oppure attraverso periferiche desktop standard (tastiera e mouse) durante l'uso in emulazione su Meta XR Simulator o Unity Editor.

### 5.3.1 Mappatura delle Azioni di Scansione e Controllo

Le azioni dell'operatore sono formalizzate in tre comandi operativi:

1. **Scansione Puntuale:** Innesca la cattura immediata del fotogramma e l'avvio della pipeline asincrona di trasmissione NDJSON;
2. **Commutazione Scansione Automatica:** Abilita o disabilita il trigger temporizzato continuo a intervalli regolari ( $\text{scanPauseSeconds} = 2.0s$ );

3. **Visualizzazione Console Log:** Commuta l’HUD nello stato esteso di visualizzazione degli errori e degli eventi.

I comandi operativi sono riportati nella seguente tabella 5.1.

| Comando Funzionale | Tastiera (PC / Simulator) | Touch Controller           | Hand Tracking (Gesto)                   |
|--------------------|---------------------------|----------------------------|-----------------------------------------|
| ScanNow            | Space / Alpha3            | Pulsante A (Button . One)  | Pizzico Dito Medio Destro               |
| ToggleAutoScan     | N / Alpha4                | Pulsante B (Button . Two)  | Pizzico Dito Medio Sinistro             |
| ToggleLog          | L                         | Pulsante Y (Button . Four) | Pizzico Dito Anulare (Entrambe le mani) |

Tabella 5.1: Matrice di normalizzazione degli input di sistema implementata in DyrInput.

### 5.3.2 Gestione dell’Hand Tracking e Isolamento delle Gesture di Sistema

L’integrazione dell’hand tracking introduce una complessa problematica di collisione con le gesture native del sistema operativo Horizon OS. Nativamente, la chiusura a contatto tra la punta del dito indice e il pollice è intercettata a basso livello dal compositor di sistema per emulare il click e aprire il menù principale di Meta. Per superare tali limitazioni, il sistema interroga a basso livello lo stato articolare della mano:

- Il comando di scansione è associato al contatto tra pollice e dito medio sulla mano destra;
- Il toggle dell’auto-scansione è demandato al pizzico con il dito medio sinistro;
- L’apertura della console di log sfrutta il contatto con il dito anulare su qualsiasi mano.

## 6. Spatial Computing e Posizionamento AR

Il seguente capitolo illustra la trasduzione dai bounding box 2D alle coordinate 3D, l'ancoraggio su mesh MRUK, le tecniche anti-affollamento e la persistenza 6-DoF mediante Meta Spatial Anchors.

### 6.1 Dal 2D al 3D: Raycasting e Geometria Proiettiva

Per quanto riguarda la conversione dei bounding box 2D, generati dal modello di visione in posizioni tridimensionali coerenti all'interno dello spazio fisico dell'utente, la pipeline di posizionamento deve affrontare due problematiche geometriche: l'inversione della proiezione prospettica e la disallineazione temporale dovuta alla latenza computazionale del modello linguistico.

#### 6.1.1 Proiezione Conforme e Disaccoppiamento Temporale

Tra il momento in cui il sensore cattura il fotogramma e l'istante in cui il backend restituisce il flusso NDJSON con le coordinate del box intercorrono tipicamente da 1 a 4 secondi. Se il raggio proiettivo venisse tracciato a partire dalla posa istantanea della testa dell'operatore al momento della ricezione del pacchetto di rete, qualsiasi rotazione o traslazione avvenuta durante l'attesa proietterebbe l'etichetta in una direzione scorretta dello spazio. Per garantire la convergenza geometrica, il sistema adotta la struttura immutabile. Essa congela la matrice estrinseca di posa della camera ( $\mathbf{p}_{\text{cap}} \in \mathbb{R}^3$ ,  $\mathbf{q}_{\text{cap}} \in \mathbb{H}$ ) e i parametri intrinseci (Field of View verticale  $\theta_v$  e aspect ratio  $\alpha$ ) nell'esatto frame di cattura. Il raggio proiettivo generato per un generico punto normalizzato dello spazio immagine  $(u, v) \in [0, 1]^2$  viene calcolato invertendo la proiezione pinhole.

#### 6.1.2 Campionamento a Corone Concentriche e Votazione a Maggioranza

L'intersezione tra un singolo raggio centrale e la geometria della stanza è soggetta a falsi positivi, se il centro geometrico del box corrisponde a una cavità dell'oggetto, il raggio attraversa il bersaglio e impatta sulla parete retrostante, collocando l'etichetta lontana dall'oggetto reale. Per risolvere tale ambiguità, viene implementato un algoritmo di campionamento multi-raggio distribuito su 13 punti normalizzati, organizzati secondo una topologia a corone concentriche:

- **Centro Geometrico:** (0.50, 0.50), campionato per primo per fungere da punto preferenziale in caso di parità;
- **Anello Stretto (8 campioni):** quattro campioni diagonali interni ( $\pm 0.15$ ,  $\pm 0.10$ ) e quattro campioni assiali ( $0$ ,  $\pm 0.20$ ) e ( $\pm 0.20$ ,  $0$ ), per intercettare l'oggetto qualora il bounding box fornito dall'MLLM sia lasco;
- **Anello Largo (4 campioni):** campioni distribuiti a ridosso dei bordi ( $\pm 0.30$ ,  $\pm 0.15$ ) per rilevare elementi estesi che occupano l'intera area delimitata.

Data la discrepanza tra le coordinate dell'immagine fornite dal modello (origine in alto a sinistra, asse  $Y$  rivolto verso il basso) e lo spazio Viewport di Unity (origine in basso a sinistra, asse  $Y$  rivolto verso l'alto), ciascun campione  $\mathbf{s}_k = (s_{x,k}, s_{y,k})$  viene mappato attraverso la trasformazione:

$$u_{vp,k} = x_0 + s_{x,k} \cdot W_{\text{box}} \quad (6.1)$$

$$v_{vp,k} = 1.0 - (y_0 + s_{y,k} \cdot H_{\text{box}}) \quad (6.2)$$

dove  $W_{\text{box}} = x_1 - x_0$  e  $H_{\text{box}} = y_1 - y_0$ . Per ogni raggio proiettato nello spazio, il sistema registra il punto d'impatto tridimensionale  $\mathbf{p}_k$  e il riferimento al trasformatore della superficie colpita  $T_k = \text{hitObject}_k$ . La scelta dell'entità reale è regolata da un principio di *majority voting*:

$$\text{Votes}(T) = \sum_{k=1}^{13} \mathbb{I}(T_k = T), \quad T_{\text{winner}} = \arg \max_T \text{Votes}(T) \quad (6.3)$$

dove  $\mathbb{I}$  è la funzione indicatrice. Il punto di ancoraggio finale  $\mathbf{p}_{\text{final}}$  viene selezionato estraendo il campione a indice minore (ovvero il più vicino al centroide geometrico) tra quelli che hanno intersecato il componente vincente  $T_{\text{winner}}$ . Questo processo richiede un tempo di esecuzione trascurabile (nell'ordine dei microsecondi per frame).

## 6.2 Integrazione con Meta MRUK (Mixed Reality Utility Kit)

Le interazioni a basso livello con l'ecosistema Meta XR SDK vengono incapsulate, isolando le chiamate native condizionate da *DYR\_META\_SDK*. Tale astrazione garantisce che l'applicativo compili ed operi sia su postazioni desktop prive di visore sia a bordo dei visori Meta Quest 3/3S.

### 6.2.1 Scene Understanding e Global Mesh

La portata massima del raggio è stata estesa a quattro volte la distanza di fallback:

$$d_{\text{reach}} = 4.0 \cdot d_{\text{fallback}} = 4.0 \cdot 2.0 \text{ m} = 8.0 \text{ metri} \quad (6.4)$$

La risoluzione dell'impatto geometrico segue una gerarchia a tre stadi:

1. **Raycasting Semantico MRUK:** Se una stanza fisica è caricata nel visore, il metodo invoca *MRU-KRoom.Raycast(ray, reach, out mrukHit, out mrukAnchor)*. Il runtime restituisce il punto d'impatto esatto sulla superficie rilevata e l'ancora semantica associata (*MRUKAnchor*).
2. **Fallback su Physics Raycast:** Se MRUK non intercetta superfici (o in modalità simulata/VR), viene eseguito un raycast fisico su scala mondo. Per scongiurare l'ancoraggio a trasformatori intermedi di scena, il puntatore all'oggetto viene estratto esplicitamente dal collider intercettato.
3. **Proiezione nel Vuoto:** Qualora nessun raggio incontri superfici reali entro 8 metri, il punto viene collocato arbitrariamente lungo la traiettoria ottica alla distanza di sicurezza prefissata

### 6.2.2 Analisi della Configurazione dell'Ambiente e Scrutinio del Background

All'inizializzazione della sessione, vengono esaminati i descrittori spaziali memorizzati dall'headset attraverso il sistema di scansione d'ambiente. Come illustrato in Tabella 6.1, la categorizzazione semantica di Meta riconosce circa 18 classi tassonomiche di arredi primari.

| Tipologia Superficie          | Comportamento di Ancoraggio e Limitazioni                                                                                                                                                                          |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MRUK Semantic Anchors</b>  | Superfici perimetrali ed elementi macroscopici ( <i>Floor, Wall Face, Ceiling, Table, Couch, Storage, Door/Window Frame</i> ). Consentono l'ancoraggio diretto al nodo trasformatore.                              |
| <b>MRUK Global Mesh</b>       | Mesh volumetrica poligonale continua ad alta densità generata dal sensore di profondità time-of-flight (ToF). Consente il raycasting su elementi minuti privi di categoria (tazze, attrezzi, componenti hardware). |
| <b>Fallback Depth (2.0 m)</b> | Attivato in assenza di mesh globale per oggetti non inclusi nelle 18 categorie semantiche. L'etichetta fluttua nel vuoto alla distanza convenzionale $d_{\text{fallback}}$ .                                       |

Tabella 6.1: Regimi di risoluzione geometrica forniti dall'integrazione con Meta MRUK.

## 6.3 Generazione delle Etichette

La rappresentazione visiva dei metadati semantici generati dall'MLLM è delegata a una classe progettata per istanziare l'interfaccia interamente via codice, garantendo il completo disaccoppiamento da prefab serializzati.

### 6.3.1 Costruzione dell'Interfaccia Olografica

Nel regime di funzionamento tridimensionale, viene istanziato un nuovo GameObject. La superficie interattiva poggia su una componente *Canvas*, i cui parametri dimensionali sono calibrati metricamente:

- **Fattore di Scala:** `WorldCanvasScale` = 0.0012, convertendo la proporzione nativa di 1 pixel UI  $\equiv$  1.2 mm nel mondo reale;
- **Dimensioni del Rettangolo UI:** larghezza fissa a 380 pixel (0.456 metri) e altezza modulare basata sul contenuto;
- **Colorazione Deterministica:** per distinguere le entità adiacenti preservando la consistenza cromatica tra sessioni, il colore del pannello viene generato in maniera deterministica. La tonalità HSV risultante è vincolata a saturazione costante (0.68), luminosità elevata (0.92) e opacità di sfondo all'88% ( $\alpha = 0.88$ ).

Per identificare lo stato, il prefisso *NEW* notifica una nuova entità documentata per la prima volta, mentre il marcatore *SEEN* segnala la re-identificazione di un elemento già censito nel grafo.

### 6.3.2 Orientamento Dinamico Verticale

Nei contesti Mixed Reality con operatore in movimento, le etichette tridimensionali devono orientarsi verso l'utente per rimanere costantemente leggibili. Tuttavia, quando l'operatore inclina la testa lateralmente (*head roll*), le etichette ruotano in sincronia, degradando la leggibilità del testo e inducendo affaticamento visivo. Per ovviare a questo difetto, viene implementato un allineamento sull'asse azimuth (*Y*).

Così facendo il piano del testo mantiene un assetto rigidamente verticale rispetto al pavimento, consentendo una lettura naturale indipendentemente dalle oscillazioni della testa.

### 6.3.3 Gestione dell'Affollamento e Impilamento Verticale

Quando più componenti fisici occupano posizioni ravvicinate nello spazio, le etichette corrispondenti tenderebbero a compenetrarsi. Per prevenire tale sovrapposizione viene calcolata preventivamente la densità locale dei marker:

$$\|\Delta \mathbf{p}_{xz}\|^2 = (x_a - x_b)^2 + (z_a - z_b)^2 \leq r_{\text{clearance}}^2 \quad (6.5)$$

dove il raggio di tolleranza sul piano orizzontale è configurato a  $r_{\text{clearance}} = 0.35$  metri. Se un nuovo marker ricade all'interno del cilindro di ingombro di etichette preesistenti, la coordinata verticale viene traslata verso l'alto applicando un offset progressivo.

#### 6.3.4 Inseguimento di Oggetti Manipolati

I sistemi convenzionali di Mixed Reality basati unicamente su descrittori di scena statici falliscono quando un oggetto manipolabile viene prelevato e riposizionato altrove. L'etichetta associata rimarrebbe ancorata alle coordinate spaziali originarie, fornendo un'indicazione spaziale obsoleta. Per cercare di risolvere il problema, a fronte di una re-identificazione, il sistema esegue un nuovo raycast ottico determinando la posizione corrente  $\mathbf{p}_{\text{new}}$ . Se lo spostamento misurato è inferiore alla soglia di 0.25 metri, la variazione viene interpretata come rumore geometrico di quantizzazione o instabilità del tracking, preservando l'etichetta nella posizione preesistente per evitare fenomeni di *jittering*.

#### 6.3.5 Culling Tronco-Conico e Regolazione del Sovraffollamento

Nel ciclo di aggiornamento post-rendering il sistema calcola la visibilità prospettica delle annotazioni, un'etichetta viene considerata attiva solo se la distanza metrica soddisfa  $\|\mathbf{p}_{\text{label}} - \mathbf{p}_{\text{head}}\| \leq d_{\text{max}} = 30.0$  metri e le coordinate di proiezione normalizzate rientrano nel tronco di vista. Qualora venga imposto un limite al numero massimo di annotazioni simultanee, le etichette candidate vengono ordinate in base al quadrato della distanza euclidea dall'osservatore.

### 6.4 Persistenza Spaziale: Meta Spatial Anchors

La semplice memorizzazione di coordinate cartesiane locali ( $\mathbb{R}^3$ ) nel database risulterebbe fallimentare: a ogni riavvio dell'headset o riconfigurazione della sessione OpenXR, il sistema di tracciamento posiziona l'origine del mondo virtuale in corrispondenza del punto di accensione contingente, invalidando la corrispondenza metrica con l'ambiente reale. Per garantire la persistenza assoluta a 6 gradi di libertà (6-DoF), l'architettura implementa la persistenza basata sulle API Meta Spatial Anchors.

#### 6.4.1 Architettura del Ciclo di Vita delle Ancore

Nel funzionamento dell'SDK, chiamate concorrenti sovrapposte comporterebbero la corruzione dei callback associati alle singole entità. Per garantire la consistenza, vengono serializzate le operazioni attraverso una coda FIFO.

Il flusso operativo di generazione si articola nei seguenti passaggi sequenziali:

1. **Richiesta di Istanziamento:** La tupla coordinate-callback viene accodata. Se il motore non ha processi in pending, estrae la richiesta e associa un template invisibile preallocato convenzionalmente a quota  $y = -1000$  metri.
2. **Generazione e Persistenza Nascosta nell'OS:** Il runtime Horizon OS attribuisce un identificativo universale univoco a 128 bit (UUID) e persiste l'ancora nel database del firmware a bordo del visore.

## 7. Strumenti per l'Editor Unity e Automazione del Setup

In questo capitolo vengono descritti gli strumenti sviluppati per ridurre il lavoro manuale di configurazione.

### 7.1 Inquadramento e Motivazione: Riduzione del Lavoro Manuale

La fase di configurazione dell'ambiente di authoring e della pipeline di compilazione rappresenta uno dei principali colli di bottiglia e una frequente sorgente di anomalie. La corretta integrazione di Meta XR SDK, MRUK (*Mixed Reality Utility Kit*), Spatial Anchors, permessi a basso livello del sistema operativo Android/Horizon OS e comunicazioni di rete locali non crittografate richiede molti passaggi manuali all'interno dell'Editor Unity e nei file di configurazione del progetto. Il progetto implementa un insieme di strumenti per l'Editor Unity e post-processor di build automatici. Questi componenti automatizzano integralmente:

- La creazione procedurale e la validazione strutturale della scena tridimensionale;
- L'iniezione dei metadati, dei permessi hardware e delle eccezioni di sicurezza a livello di manifest e configurazione Gradle durante la compilazione dell'APK.

### 7.2 Generazione Automatica e Validazione della Scena

Viene fornito all'operatore un set di comandi dedicati, accessibili direttamente dalla barra dei menù di Unity, per assemblare, aggiornare e collaudare la scena applicativa senza richiedere alcun trascinarsi manuale di prefab o interventi manuali nell'Inspector.

#### 7.2.1 Pipeline di Creazione Deterministica della Scena

Invocando la voce di menù `Tools > DYR > Create Scene` (`CreateScene()`), lo script esegue la costruzione deterministica della scena, salvandola nel percorso convenzionale `Assets/Scenes/DocumentYourReality.unity` e configurandola come primo elemento negli `EditorBuildSettings`:

1. **Risoluzione e Configurazione dell'OVRCameraRig:** Viene effettuata una ricerca tra gli asset di progetto e all'interno dei pacchetti installati (`com.meta.xr.sdk.core`) per localizzare il prefab `OVRCameraRig`. Il sistema istanzia `OVRCameraRig`, tagga `CenterEyeAnchor` come `MainCamera` e de-tagga `LeftEyeAnchor`. Con fallback a camera semplice (`y=1.6`) se l'SDK Meta è assente.
2. **Configurazione Globale dell'Ecosistema Meta:** Lo script attiva programmaticamente i flag di sistema:

- `insightPassthroughEnabled = true`: abilita il layer di composizione stereoscopica del mondo reale;
  - `isPassthroughCameraAccessEnabled = true`: sblocca a basso livello l'accesso alle telecamere raw per l'invio all'MLLM.
3. **Integrazione Geometrica con Meta MRUK**: Ricerca e istanzia il prefab radice MRUK.
  4. **Allestimento della Stanza Virtuale per Test**: Istanzia il prefab dell'arredo sintetico *LowPolyLivingRoom-Pack\_Demo*, disattivandolo preventivamente (*SetActive(false)*). In questo modo la stanza virtuale rimane silente durante l'uso in Passthrough reale, venendo abilitata esclusivamente se l'operatore seleziona la modalità Vr.
  5. **Taratura dell'Illuminazione ad Alte Prestazioni**: Aggiunge una luce direzionale ruotata con angoli di Eulero ( $50^\circ$ ,  $-30^\circ$ ,  $0^\circ$ ) e intensità 1.1.
  6. **Istanziamento del Controller Centrale**: Crea l'oggetto *DocumentReality*.

## 7.3 Automazione del Manifest Android e Build Pipeline

La compilazione di un'applicazione Mixed Reality destinata ad Horizon OS comporta la generazione di un package Android (.APK). A partire da Android 9 (API level 28) e con il progressivo inasprimento introdotto da Android 11+ in merito allo *Scoped Storage*, le policy predefinite del sistema operativo impediscono le chiamate verso server HTTP non cifrati e blindano l'accesso ai sensori ottici e al filesystem condiviso. Per eliminare la necessità di modificare manualmente il file *AndroidManifest.xml* ad ogni rigenerazione del progetto Gradle, il sistema implementa la classe *DyrAndroidManifestPostprocessor*.

### 7.3.1 Risoluzione del Traffico in Chiaro LAN

A partire da Android 9 (API 28), la piattaforma impone di default l'obbligo di crittografia TLS su tutte le comunicazioni socket, rifiutando a priori il traffico in chiaro verso indirizzi `http://`. Qualsiasi richiesta HTTP multipart o streaming NDJSON verrebbe terminata istantaneamente dal layer di sicurezza del sistema operativo Android con un errore di rete di tipo *"Cleartext HTTP traffic not permitted"*. Il metodo `EnsureCleartextTraffic` interviene sul nodo radice `<application>` inserendo programmaticamente l'attributo:

```
android:usesCleartextTraffic = "true" (7.1)
```

## 8. Valutazione e Risultati

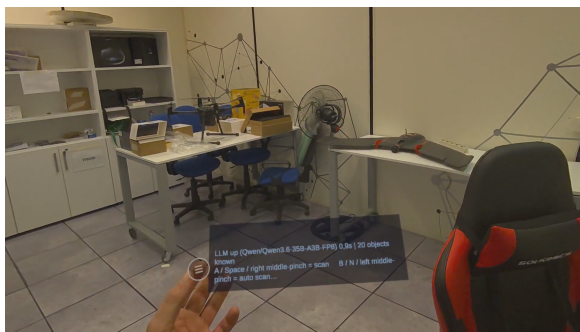
In questo capitolo vengono presentati i risultati sperimentali derivanti dalla validazione del sistema nei tre ambienti operativi previsti (Passthrough su visore fisico, flussi Video preregistrati e ambiente sintetico VR).

### 8.1 Analisi Qualitativa nelle Diverse Modalità Operative

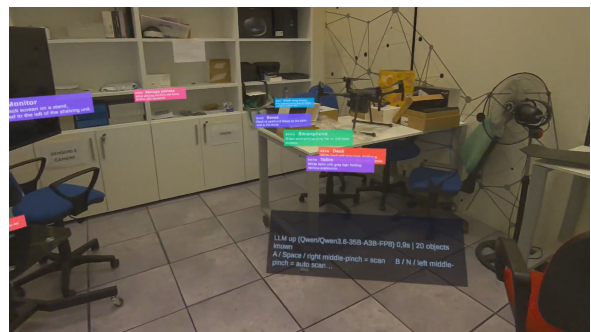
Il sistema è stato sottoposto a test per verificarne l'usabilità, la fedeltà del grounding visivo e la correttezza della generazione documentale.

#### 8.1.1 Modalità Passthrough su Visore Reale (Meta Quest 3)

La validazione principale è avvenuta tramite il visore nel laboratorio, caratterizzato dalla compresenza di diverse tipologie di oggetti.



(a) Innesco della scansione tramite gesture della mano.



(b) Proiezione delle etichette nello spazio 3D.

Figura 8.1: Sequenza temporale di rilevamento in Passthrough: dal trigger gestuale all'istanziatura progressiva delle etichette ancorate agli oggetti fisici.

Le Figure 8.1a e 8.1b illustrano la dinamica sequenziale di scansione:

1. **Innesco e Diagnostica:** L'operatore innesca la cattura mediante la gesture di pinch tra pollice e dito medio destro (visibile in basso a sinistra in Figura 8.1a), visualizzando sull'HUD flottante lo stato di connettività verso il modello linguistico (*LLM up (Qwen/Qwen3.6-35B-A3B-FP8) 0,9s | 20 objects known*).
2. **Ricezione Progressiva e Proiezione:** Entro pochi secondi dall'invio del frame compresso, il flusso NDJ-SON restituisce le coordinate dei bounding box. Il client esegue il campionamento multi-raggio e genera le etichette tridimensionali visibili in Figura 8.1b, collocandole stabilmente al di sopra di arredi, monitor e tavoli.

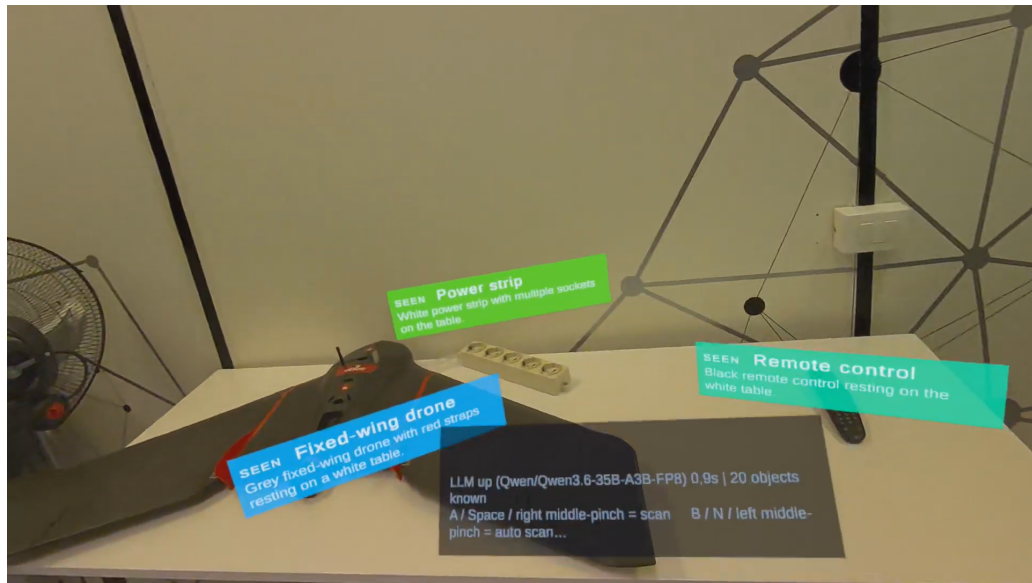


Figura 8.2: Dettaglio ravvicinato delle etichette semantiche sul banco operativo.

**Comprensione Relazionale e Contestualizzazione Spaziale** Un risultato di rilievo, evidenziato dal dettaglio ravvicinato in Figura 8.2, riguarda la natura delle descrizioni generate dal modello multimodale. L'MLLM non si limita alla classificazione decontestualizzata dell'oggetto atomico, ma incorpora esplicitamente le relazioni spaziali con gli altri oggetti:

- **Fixed-wing drone:** *"Grey fixed-wing drone with red straps resting on a white table."*
- **Power strip:** *"White power strip with multiple sockets on the table."*
- **Remote control:** *"Black remote control resting on the white table."*

Tale caratteristica fornisce all'operatore subentrante un riferimento spaziale immediato, per discriminare componenti identici dislocati su piani differenti o in configurazioni operative complesse.

### 8.1.2 Modalità Video Offline

La pipeline di ingestione video su sequenze MP4 registrate è stata testata su scene domestiche reali (Figure 8.3a e 8.3b).

Come si evince dall'HUD diagnostico a schermo, il sistema elabora i fotogrammi identificando con successo elementi quali *Television*, *Remote control*, *Standing fan* e *Media console*.

### 8.1.3 Modalità VR Sintetica (Ambiente 3D Simulato)

L'ambiente virtuale procedurale (basato su asset low-poly) ha permesso di convalidare le logiche di culling e la risoluzione di scene a elevata densità geometrica in totale assenza dell'headset fisico (Figure 8.4a e 8.4b).

Le prove condotte confermano che il culling del layer UI (*layer* = 5) impedisce qualsiasi ricorsione o artefatto di auto-osservazione. In Figura 8.4a, il modello discrimina con precisione arredi geometricamente simili ma cromaticamente distinti (*Red stool* rispetto a *Wooden stool*), assegnando loro badge *NEW* differenziati ed etichette cromaticamente consistenti.



(a) Scena iniziale.



(b) Scena etichettata.

Figura 8.3: Valutazione in modalità Video.

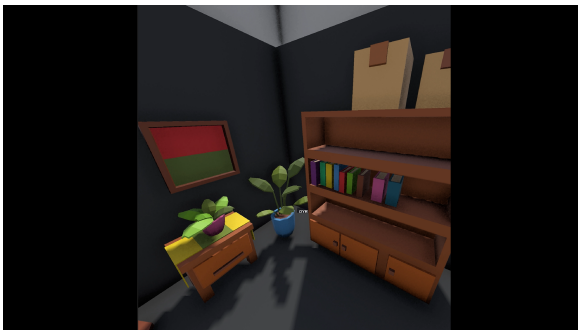


(a) Scena complessa con sedute multiple ravvicinate.



(b) Etichettatura di arredi e complementi d'arredo.

Figura 8.4: Collaudo in ambiente simulato VR: discriminazione cromatica ed etichettatura procedurale.



(a) Inquadratura prima della scansione.



(b) Etichette semantiche generate.

Figura 8.5: Confronto prima e dopo la scansione in un angolo della stanza virtuale.

Nelle Figure 8.5a e 8.5b viene mostrato il comportamento del sistema su un angolo della stanza prima e dopo l’elaborazione. Oltre a evidenziare la convivenza tra entità note (*SEEN Floor lamp*) e nuovi elementi censiti (*NEW* su *Bookshelf*, *Side table* e *Potted plant*), si nota la ricchezza descrittiva del modello, che coglie dettagli quali il colore del vaso (*“blue pot on the floor”*) o gli oggetti appoggiati sul tavolino (*“yellow top and a purple object on it”*), ancorando stabilmente i pannelli alle rispettive mesh.

## 8.2 Profilazione Quantitativa delle Latenze e Strategie di Streaming

Per quantificare l’impatto delle ottimizzazioni architetturali introdotte nel backend (trasmissione incrementale NDJSON e terminazione anticipata dello stream), è stata condotta una campagna di benchmark su un set di 10 fotogrammi estratti da un video di test. I test sono stati eseguiti impiegando il modello *Qwen3.6-35B-A3B-FP8*, misurando l’accordo qualitativo tra i metodi (risultato pari all’83%) e profilando le metriche temporali e di consumo di token riassunte in Tabella 8.1.

| Strategia di Inferenza                              | Oggetti    | TTFL         | Latenza Totale | Token Emessi |
|-----------------------------------------------------|------------|--------------|----------------|--------------|
| Due chiamate distinte (Detect + Desc)               | 6.9        | 14.0 s       | 14.0 s         | 969          |
| Singola chiamata (Monolitica)                       | 6.3        | 12.5 s       | 12.5 s         | 884          |
| Singola chiamata + Streaming NDJSON                 | 6.3        | <b>2.7 s</b> | 12.5 s         | ~632         |
| <b>NDJSON + Cap Anticipato (<math>N = 5</math>)</b> | <b>5.0</b> | <b>2.7 s</b> | <b>10.1 s</b>  | <b>~506</b>  |

Tabella 8.1: Confronto prestazionale delle strategie di inferenza su 10 frame (*Qwen3.6-35B-A3B-FP8*). TTFL indica il *Time to First Label*.

I dati sperimentali evidenziano i seguenti comportamenti:

- **Abbattimento della Latenza Percepita via NDJSON:** Nelle architetture senza streaming a blocchi, l’utente è costretto ad attendere il completamento dell’intera risposta generativa. Con l’introduzione del parser incrementale riga-per-riga, il tempo di ricezione del primo marker (*Time to First Label*, TTFL) si riduce a soli 2.7 secondi (un miglioramento del 78.4%), consentendo al visore di proiettare i primi ancoraggi con minore latenza.
- **Efficacia della Terminazione Forzata (*Latency Dial*):** L’imposizione di un tetto massimo ai rilevamenti ( $N = 5$  per frame) abbinata alla chiusura asincrona del socket HTTP consente di interrompere la computazione autoregressiva sulla GPU non appena saturati gli slot utili. Ciò abbate la latenza end-to-end da 12.5 a 10.1 secondi e comprime i token generati da circa 884 a 506, liberando in anticipo la VRAM per i frame successivi.

### 8.2.1 Studio Comparativo A/B sul Prompt: Prefill vs Decodifica

Prima di consolidare il prompt definitivo di scansione (*scan.j2*), è stato condotto un test comparativo A/B valutando una versione compressa del prompt di sistema (3254 caratteri contro i 4053 caratteri del prompt esteso, pari a una riduzione del  $-20\%$ ).

L’esito sperimentale ha dimostrato che un prompt più corto non è più veloce. Normalizzando i risultati per numero di entità identificate, le due varianti hanno fatto registrare metriche perfettamente sovrapponibili:

$$t_{\text{obj}} = 1.98 \text{ s/oggetto}, \quad C_{\text{obj}} = 140 \text{ token/oggetto} \quad (8.1)$$

Tale fenomeno è riconducibile all’architettura dei transformer e alla GPU: i token in ingresso (*prompt tokens*) vengono processati in parallelo durante la fase di *prefill*, il cui costo computazionale aggiuntivo è nell’ordine di

poche decine di millisecondi. Al contrario, la latenza di un MLLM è determinata dalla fase autoregressiva di decodifica sequenziale (generazione token per token). Accorciare le istruzioni qualitative in *scan.j2* non produce alcun beneficio tangibile di reattività, mentre rischia di degradare la precisione delle delimitazioni; l'unica leva computazionale efficace risiede nella limitazione del volume di token in output, risultato conseguito tramite lo schema Shape A e la chiusura anticipata dello stream NDJSON.

### 8.3 Validazione Empirica della Re-identificazione: Test a Doppio Passaggio

La correttezza del ciclo di memorizzazione, estrazione dei descrittori e ri-associazione delle entità è stata validata sperimentalmente sottoponendo il medesimo flusso a due passaggi di scansione sequenziali sullo stesso ambiente.

| Passaggio 1: Rilevamento Iniziale (store azzerato)                                                                                                                                | Passaggio 2: Re-identificazione (Warm)                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Il database viene preventivamente azzerato tramite l'endpoint POST <code>/objects/reset</code> . Il modello elabora la scena senza alcuna entità pregressa ( <i>cold start</i> ). | Il database conserva gli oggetti censiti nel Passaggio 1. La <i>cue list</i> viene iniettata nel prompt.                             |
| Tutti gli elementi rilevati vengono etichettati come [NEW] e registrati con identificativi univoci (obj_6a5addb7, obj_04d7289c).                                                  | Gli elementi ri-osservati vengono classificati come [SEEN], mantenendo rigorosamente i medesimi ID (obj_6a5addb7, obj_04d7289c).     |
| Vengono generati i relativi documenti JSON nella cartella <code>entities/</code> e le osservazioni in <code>annotations/</code> .                                                 | <b>Zero nuovi record creati:</b> l'algoritmo non istanzia duplicati, aggiornando unicamente il timestamp <code>last_seen_at</code> . |

Tabella 8.2: Comportamento operativo del test a doppio passaggio per la validazione della persistenza.

Come riassunto in Tabella 8.2, l'esperimento ha evidenziato l'efficacia del passaggio da Shape B a Shape A:

- Durante il **Passaggio 1**, il modello istanzia i record ex-novo emettendo descrizioni verbose, con un costo medio di circa 120 token per elemento;
- Nel **Passaggio 2**, l'iniezione della *cue list* consente al resolver di operare a livello di match esplicito (*known\_id*). Il modello emette payload Shape A ridotti a meno di 25 token.

### 8.4 Limiti dello Scene Understanding Nativo e Ruolo della Global Mesh

L'interazione con l'SDK Meta XR ha evidenziato un disallineamento strutturale tra la percezione geometrica dell'headset e la capacità semantica ad ampio vocabolario tipica di un MLLM:

1. **Vincolo delle 18 Categorie Fisse:** Il modulo di *Scene Understanding* nativo del visore riconosce un insieme chiuso di 18 classi macroscopiche (*Floor, Wall Face, Ceiling, Table, Couch, Storage, Door, Window*, ecc., oltre alla classe generica OTHER).
2. **Capacità Lessicale Aperta dell'MLLM:** Il modello di visione è in grado di discriminare con precisione anche i componenti per i quali non esiste alcun descrittore semantico predefinito nel runtime Meta a cui associare un *MRUKAnchor* nativo.

Come schematizzato nella Tabella 8.3, la corretta esecuzione della procedura preliminare di *Space Setup* sul visore (ovvero la scansione di tutte le superfici dell'ambiente per l'estrazione della *Global Mesh* continua tramite sensore Time-of-Flight) costituisce il presupposto per garantire la plausibilità fisica dell'applicativo. In assenza di mesh globale, le annotazioni relative a elementi per cui non esiste un descrittore semantico predefinito degradano a indicatori fluttuanti a distanza prefissata.

| Regime di Rilevamento                                                       | Meccanismo di Ancoraggio                                       | Plausibilità e Comportamento Fisico                                                                                                     |
|-----------------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Categoria MRUK Nota</b>                                                  | MRUKAnchor dedicato                                            | L'etichetta si aggancia direttamente all'oggetto.                                                                                       |
| <b>Oggetto privo di descrittore semantico predefinito CON Global Mesh</b>   | Collisione raycast su mesh poligonale ToF                      | <b>Comportamento ottimale:</b> l'etichetta poggia perfettamente sulla superficie fisica del componente non classificato.                |
| <b>Oggetto privo di descrittore semantico predefinito SENZA Global Mesh</b> | Fallback Depth fisso ( $d_{\text{fallback}} = 2.0 \text{ m}$ ) | <b>Comportamento non ottimale:</b> l'etichetta resta sospesa nel vuoto lungo la linea di vista, priva di un vincolo superficiale reale. |

Tabella 8.3: Matrice di comportamento del posizionamento AR in funzione della configurazione dello spazio.

## 9. Conclusioni e Sviluppi Futuri

Quest'ultimo capitolo riassume gli obiettivi conseguiti, discute i vincoli emersi e delinea le prospettive future di sviluppo

### 9.1 Sintesi dei Risultati Conseguiti

Il presente lavoro ha portato alla progettazione, implementazione e validazione sperimentale di Document Your Reality.

Attraverso l'integrazione di un frontend sviluppato in Unity URP su piattaforma Meta Horizon OS e un backend asincrono in Python FastAPI, sono stati conseguiti gli obiettivi prefissati:

- **Grounding Visivo e Descrizione Contestualizzata:** Il sistema converte flussi video reali (da telecamere passthrough del visore o da sequenze registrate) in descrizioni olografiche. È stato dimostrato come il modello MLLM non si limiti alla catalogazione superficiale del target, ma ne incorpori le relazioni spaziali con gli altri elementi presenti nell'ambiente.
- **Ottimizzazione della Latenza Generativa:** L'implementazione del protocollo NDJSON e del meccanismo di *early-truncation* ha consentito di abbattere il tempo percepito per il primo rilevamento (TTFL) a soli 2.7 secondi.
- **Risoluzione dell'Identità e Persistenza a Lungo Termine:** Il modello di prompt gerarchico a doppio schema (Shape A e Shape B), supportato dall'architettura *MemoryStore* e dall'aggiornamento incrementale di *EntityMemory*, ha garantito il successo nei test a doppio passaggio, azzerando la creazione di record duplicati e riducendo il numero di token per entità da 120 a meno di 25.
- **Plausibilità Geometrica:** La combinazione del campionamento multi-raggio a corone concentriche (13 punti con votazione a maggioranza) e dell'ancoraggio su Meta Spatial Anchors ha consentito di superare i limiti del tracking tra sessioni distinte.

### 9.2 Problematiche Rilevate: Il Conflitto Architetturale con Vuforia

Durante le prime fasi di sviluppo e concezione del sistema, è stata valutata l'adozione del framework **PTC Vuforia Engine** per gestire il compito di tracking e ancoraggio 2D/3D tramite marcatori visuali (*Image Targets* e *Model Targets*). Tuttavia, l'analisi architetturale e i primi test di integrazione all'interno del motore Unity hanno evidenziato un'incompatibilità strutturale non trascurabile con l'ecosistema Meta XR.

**Collisione nell'Accesso al Flusso Video** Il problema principale è dovuto alla necessità di entrambi di avere il controllo esclusivo sul flusso video dell'headset:

- L'infrastruttura di tracciamento di Meta XR opera attorno alla gerarchia dell'*OVRCameraRig*, gestendo il passthrough interfacciato tramite il componente *PassthroughCameraAccess* dell'SDK, il cui accesso ai sensori è subordinato al permesso Android protetto *horizonos.permission.HEADSET\_CAMERA*;
- Il framework Vuforia, per operare il riconoscimento geometrico delle feature planari, richiede a sua volta il controllo esclusivo del flusso visivo mediante il componente proprietario *ARCamera*.

L'istanziatura contemporanea dei due moduli all'interno del medesimo grafo di scena provoca un conflitto di risorsa (*camera hardware contention*) nel livello di astrazione del kernel Android. Tale condizione generava sistematicamente il blocco dei driver video, la disattivazione del passthrough stereoscopico e il fallimento del compositor di sistema di Horizon OS.

A fronte di tale vincolo, si è scelto di escludere l'SDK Vuforia dall'architettura, virando verso una soluzione basata sulla combinazione tra le primitive geometriche di Meta MRUK / Meta Spatial Anchors per il posizionamento 6-DoF e la flessibilità ad ampio vocabolario del modello multimodale MLLM per il riconoscimento dell'identità visiva degli oggetti.

## 9.3 Sviluppi Futuri

I possibili sviluppi futuri si articolano lungo quattro direttrici principali:

### 9.3.1 Inferenza On-Device tramite Modelli Quantizzati (Edge AI)

L'attuale architettura vincola il visore alla presenza di un server locale raggiungibile via rete Wi-Fi. La rapida evoluzione dei modelli linguistici-visivi compatti (quali la serie SmolVLM o varianti 2B/4B di Gemma e Qwen) apre la strada all'esecuzione dell'inferenza direttamente a bordo del visore. Sfruttando i framework di quantizzazione a 4-bit (es. Qualcomm AI Hub o ONNX Runtime Execution Provider su architettura NPU Snapdragon XR2 Gen 2), sarà possibile rendere il visore completamente autonomo, eliminando la dipendenza dal backend di rete.

### 9.3.2 Sessioni Collaborative Multi-Utente su Cloud Anchors

La condivisione delle ancore spaziali è attualmente circoscritta all'archivio locale del singolo visore: l'UUID viene persistito nel backend, ma resta risolvibile unicamente dal dispositivo che lo ha generato. Un'evoluzione naturale potrebbe consistere nell'adozione delle API *Meta Shared Spatial Anchors* o di *Azure Spatial Anchors*. Tale estensione permetterà a più utenti di operare all'interno del medesimo spazio fisico simultaneamente, visualizzando e modificando in tempo reale le etichette semantiche generate dagli altri operatori.

### 9.3.3 Interazione Vocale Multimodale Bidirezionale (VUI)

L'attuale interfaccia utente si basa sull'interazione visiva e su gesture con le dita. L'integrazione di un motore *Speech-to-Text* consentirebbe di instaurare un dialogo in linguaggio naturale tra operatore e memoria semantica dell'ambiente. L'operatore potrebbe porre domande contestuali ad alta voce e vedere evidenziati nello spazio 3D unicamente i componenti pertinenti alla richiesta.

#### **9.3.4 Inseguimento Dinamico 6-DoF di Oggetti Manipolati**

Sebbene il sistema sia in grado di aggiornare la posizione dell'etichetta al re-inquadramento di un oggetto spostato, l'ancoraggio durante la fase di manipolazione attiva non è continuo. L'accoppiamento della re-identificazione MLLM con algoritmi di tracciamento basati su flusso ottico 3D consentirebbe di mantenere l'etichetta agganciata all'oggetto anche mentre l'operatore lo trasporta da una stanza all'altra.